

ASL DISTRIBUTION · DATA & ANALYTICS

Dimensional Data Modelling

Star and Snowflake Schemas

Why we convert a single flat table into a fact-and-dimension model — and when we should not.

Prepared for: **Bhavik Gandhi**

Date: 15 August 2026

Version: 1.0 · Internal reference document

Contents

1. Purpose of this document	3
Who this is for	3
2. The starting point: one big flat table	4
3. What a star schema is	5
3.1 The fact table	7
3.2 The dimension tables	8
3.3 The correct mental model	9
4. Grain — the first decision you make	10
5. The four-step design method	11
6. Why convert — the reasons in full	12
6.1 Redundancy and storage	12
6.2 Consistency and a single definition	12
6.3 History — slowly changing dimensions	13
6.4 Conformed dimensions across multiple facts	14
6.5 Maintainability and update anomalies	14
6.6 Compression and query performance	14
6.7 A clean semantic model for business users	15
6.8 Scalability	15
7. Types of fact table	16
8. The snowflake schema	17
8.1 When snowflaking is justified	17
8.2 When it is not	18
9. Star vs. snowflake vs. one big table	19
10. The same question, both ways	20
11. Choosing a model	21
12. How the two approaches fit together	22
13. Common mistakes checklist	23
14. Glossary	24

1. Purpose of this document

This document answers a question that every analyst asks the first time they meet a data warehouse:

The question If a single flat table already contains every column I need — item, order, date, address, customer, revenue, vendor — why would I break it apart into a fact table and dimension tables, only to join them back together every time I run a query?

The short answer is that a star schema is **not** created because the flat table lacks information. It is created because the flat table lacks **structure**. Structure is what keeps the numbers correct as the data grows, as attributes change, as new subject areas are added, and as more people start querying it.

The document works through the reasoning using a sales example, defines the terminology precisely, and finishes with guidance on when a flat table is in fact the right answer.

Who this is for

- Analysts and BI developers building models in Power BI, Tableau or SQL.
- Data engineers designing warehouse layers.
- Anyone who has inherited a wide flat extract and been asked to "model it properly".

2. The starting point: one big flat table

Below is the table as it usually arrives from a source system or a spreadsheet export. Every column that anyone might need is present on every row.

■ ONE BIG FLAT TABLE — ORDERS

ORDER_ID	ITEM_ID	ORDER_DATE	CUSTOMER_ID	CUSTOMER_CITY	VENDOR_ID	VENDOR_NAME	VENDOR_COUNTRY	REVENUE
0001	I101	2026-08-01	C001	Toronto	V01	ABC Logistics	Canada	100.00
0002	I102	2026-08-01	C001	Toronto	V01	ABC Logistics	Canada	200.00
0003	I103	2026-08-02	C002	Mississauga	V02	XYZ Freight	Canada	150.00
0004	I101	2026-08-02	C001	Toronto	V01	ABC Logistics	Canada	120.00
0005	I104	2026-08-03	C001	Toronto	V01	ABC Logistics	Canada	340.00

Highlighted cells are descriptive text repeated on every row — across 100 million rows this is the same string stored 100 million times.

Figure 1 — A flat "one big table" extract of order data.

This table works. The following query runs, and returns a correct answer today:

```
SELECT Vendor_Name, SUM(Revenue) FROM Orders GROUP BY Vendor_Name;
```

So the objection is a fair one. The problems with this table are not visible at five rows. They appear at five hundred million rows, at the second year of history, and at the second team that needs to use it. The rest of this document is a catalogue of those problems and how the star schema solves each one.

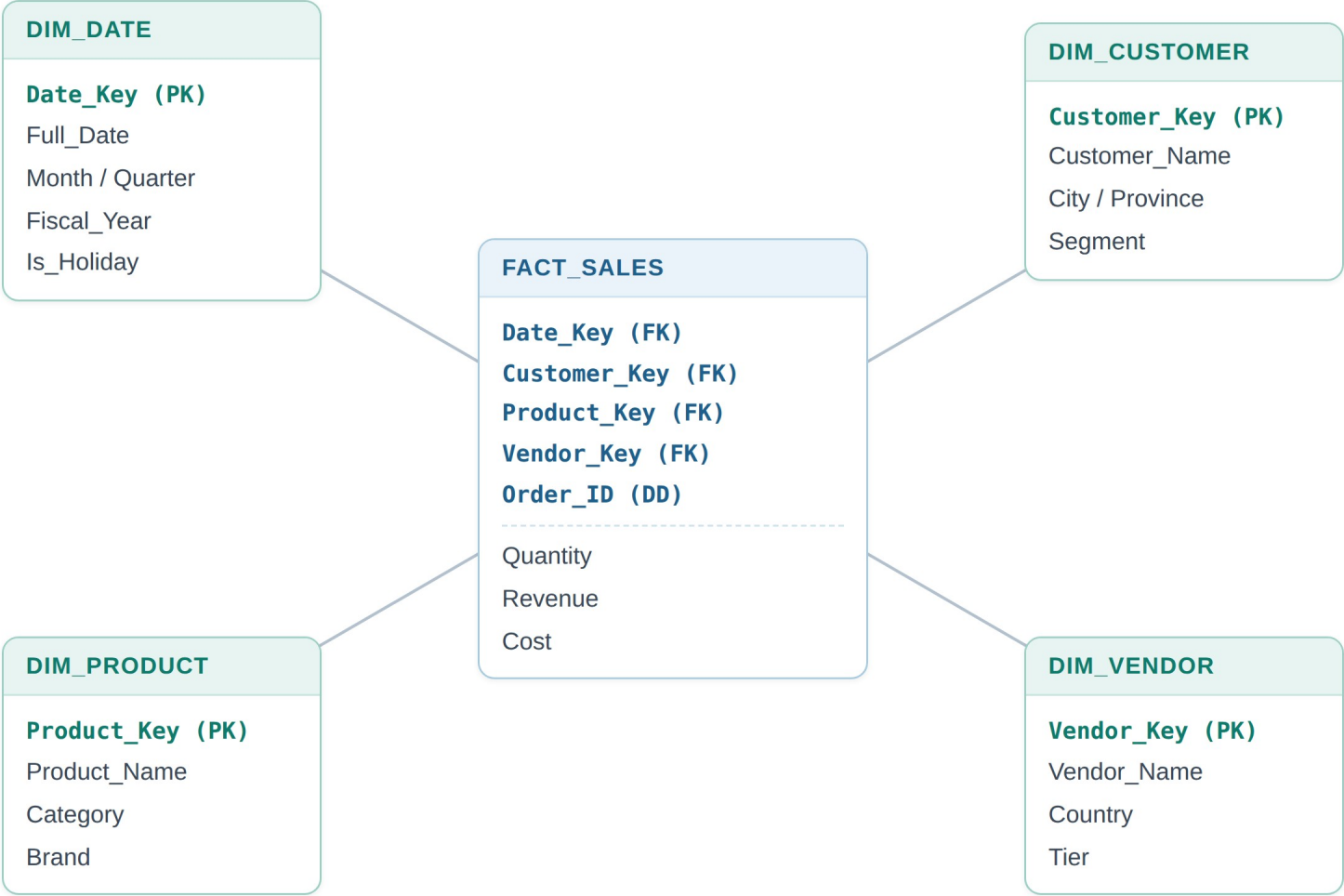
3. What a star schema is

A star schema splits the flat table into two kinds of table, joined by keys:

- **One fact table** at the centre, holding the measurements of a business event.
- **Several dimension tables** around it, holding the descriptive context of that event.

Drawn out, the fact sits in the middle and the dimensions radiate from it — which is where the name comes from.

■ STAR SCHEMA



■ Fact — measures & keys ■ Dimension — descriptive attributes PK = primary key · FK = foreign key · DD = degenerate dimension

Figure 2 — A star schema. One fact table, four dimensions, one join hop to each.

3.1 The fact table

The fact table records **what happened**. It is narrow and extremely tall: a handful of foreign keys, a handful of numeric measures, and one row for every event.

■ FACT_SALES — GRAIN: ONE ROW PER ORDER LINE

ORDER_ID (DD)	DATE_KEY	CUSTOMER_KEY	PRODUCT_KEY	VENDOR_KEY	QUANTITY	REVENUE	COST
0001	20260801	4471	881	12	2	100.00	62.00
0002	20260801	4471	902	12	4	200.00	131.00
0003	20260802	4472	915	27	1	150.00	94.50
0004	20260802	4471	881	12	3	120.00	74.40
0005	20260803	4471	928	12	5	340.00	210.80

Narrow, tall, mostly integers. Order_ID stays here as a **degenerate dimension** — an identifier with no attributes of its own.

Figure 3 — Fact_Sales. Note that the descriptive text is gone; only keys and measures remain.

Two details worth noticing:

- **Surrogate keys.** The fact stores `Customer_Key = 4471`, an integer generated by the warehouse, rather than the source system's `C001`. Surrogate keys are compact, immutable, and — critically — allow the same customer to have several versions over time (see section 6.3).
- **Degenerate dimensions.** `Order_ID` stays in the fact table even though it is text. It is an identifier used for grouping and drill-through, and it has no attributes of its own, so there is no dimension for it to live in. This is called a degenerate dimension and it is entirely normal.

3.2 The dimension tables

Dimensions record **who, what, when, where and which**. They are wide and short: many descriptive columns, comparatively few rows.

■ DIMENSION TABLES — SMALL, WIDE, DESCRIPTIVE

Dim_Customer

CUSTOMER_KEY	CUSTOMER_ID	CUSTOMER_NAME	CITY	PROVINCE	SEGMENT
4471	C001	Northline Retail	Toronto	ON	Enterprise
4472	C002	Peel Supply Co.	Mississauga	ON	Mid-Market
4473	C003	Harbour Goods	Halifax	NS	SMB

Dim_Vendor

VENDOR_KEY	VENDOR_ID	VENDOR_NAME	COUNTRY	TIER
12	V01	ABC Logistics	Canada	Preferred
27	V02	XYZ Freight	Canada	Standard
31	V03	Delta Cartage	USA	Standard

Thousands of rows, not billions. One authoritative spelling of every name. Changing “ABC Logistics” is a one-row update.

Figure 4 — Dimension tables. Thousands of rows, not billions, with one authoritative spelling of every name.

3.3 The correct mental model

A very common shortcut is to say "numbers go in the fact table, text goes in the dimension table". That rule gets the right answer often enough to feel true, and then quietly produces a bad model. The real distinction is **role**, not data type.

■ THE CORRECT MENTAL MODEL

	FACT TABLE	DIMENSION TABLE
Answers	What happened, and how much?	Who, what, when, where, which?
Contains	Measures at a defined grain + foreign keys	Descriptive attributes used to slice and group
Shape	Narrow and very tall (millions–billions of rows)	Wide and short (hundreds–millions of rows)
Data types	Mostly integers, decimals, keys	Mostly text, dates, flags — but numbers too
Used in SQL	SUM() / COUNT() / AVG()	WHERE / GROUP BY / ORDER BY
Growth	Grows with every business event	Grows only when a new entity appears
Common mistake	“All numbers go here”	“All text goes here”

A dimension can hold numbers (unit weight, credit limit, age band) — they are attributes, not additive measures. A fact can hold text (Order_ID) — it is an identifier, not a description.

Figure 5 — Fact versus dimension: the distinction is role, not data type.

In practice A dimension can happily contain numbers — unit weight, credit limit, list price, age band. Those are attributes you filter and group by, not measures you sum. Equally, a fact table can contain text, such as an order number. Ask "would I ever SUM this?" rather than "is this a number?".

4. Grain — the first decision you make

Before choosing a single column, you must be able to complete this sentence: **"One row in this fact table represents..."**

That sentence is the **grain**. It is the most important decision in the whole model, because every dimension you can attach and every measure you can store follows from it. If the grain is ambiguous, the table will double-count, and no amount of downstream SQL will fix it.

■ GRAIN — THE FIRST DECISION YOU MAKE

FACT TABLE	ONE ROW REPRESENTS...	TYPICAL MEASURES
Fact_Sales_Line	One line item on one order	Quantity, Revenue, Cost, Discount
Fact_Order_Header	One complete order	Order_Total, Shipping_Fee
Fact_Inventory_Snapshot	One product, one warehouse, one day	On_Hand_Qty, On_Hand_Value
Fact_Shipment	One carton dispatched	Weight, Freight_Cost, Days_In_Transit
Fact_Account_Balance	One account at one month end	Closing_Balance

Declare the grain in one sentence before choosing columns. If you cannot say it in one sentence, the table is not ready to build.

Figure 6 — Common fact tables and their grain.

Rule Declare the grain in one plain sentence before designing anything else. If you cannot state it in one sentence, the table is not ready to be built. Changing the grain later means rebuilding the fact table from source.

5. The four-step design method

The standard Kimball method for turning a business process into a star schema is four steps, always in this order.

■ THE FOUR-STEP DESIGN METHOD

STEP	DECISION	APPLIED TO THE ASL ORDERS EXAMPLE
1	Choose the business process	Order fulfilment — the moment revenue is recognised
2	Declare the grain	“One row per item line on a customer order”
3	Identify the dimensions	Date, Customer, Product/Item, Vendor
4	Identify the facts	Quantity, Revenue, Cost, Discount, Profit

Never skip step 2. Grain drives everything that follows, and changing it later means rebuilding the fact table.

Figure 7 — The four-step dimensional design method applied to the orders example.

Applied to the flat table in Figure 1, the outcome is exactly the star in Figure 2: a `Fact_Sales` table at order-line grain, surrounded by Date, Customer, Product and Vendor dimensions.

6. Why convert — the reasons in full

Each subsection below is a distinct problem with the flat table, and how the star schema addresses it.

6.1 Redundancy and storage

In Figure 1, the string "ABC Logistics" is stored on every single row that vendor ever touched. At 100 million order lines, that is 100 million copies of a 13-character string, plus its country, tier, address and every other vendor attribute you decide to add.

In the star schema, the fact stores a 4-byte integer `Vendor_Key = 12` and the string lives once in `Dim_Vendor`. The saving compounds with every descriptive column you add, and unlike the flat table, adding a new vendor attribute costs one column on a small table rather than one column on a billion-row table.

6.2 Consistency and a single definition

This is the reason most likely to cause a visible failure in a report. In a flat table the vendor name is typed, loaded or corrected independently on every row. It only takes one bad load, one manual fix, or one source system with different casing:

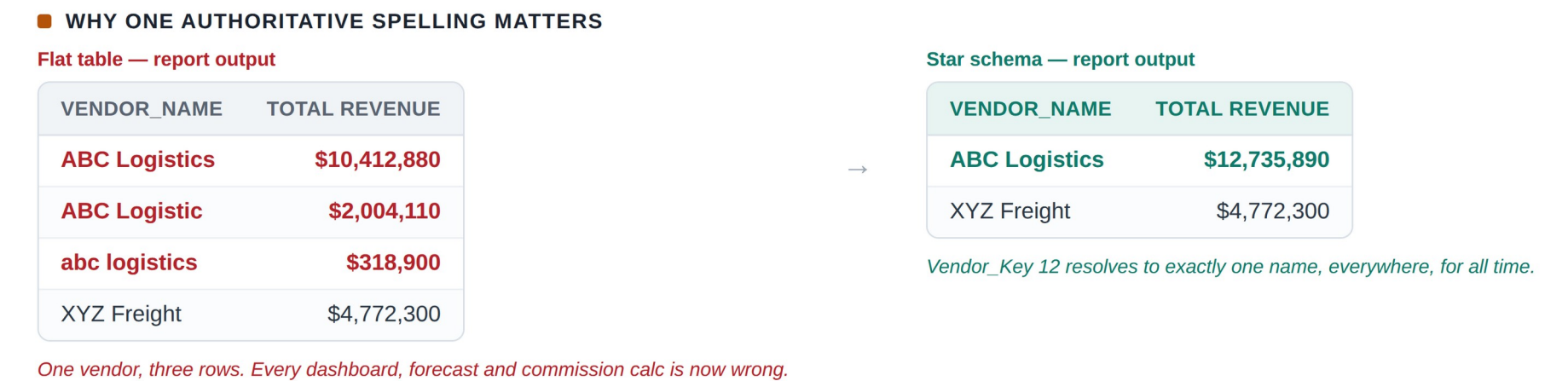


Figure 8 — The same vendor, split three ways by inconsistent text in a flat table.

With a dimension, `Vendor_Key = 12` resolves to exactly one name, in every report, for every user, for all time. Correcting a misspelling is a single-row update that instantly fixes every historical report. In a flat table it is a 20-million-row UPDATE that may half-succeed.

6.3 History — slowly changing dimensions

This is the reason a flat table cannot be made to work, no matter how wide it is.

Suppose customer C001 relocates from Toronto to Vancouver. In the flat table, what does the `Customer_City` column mean? Where they live now, or where they lived when the order was placed? You cannot store both, and whichever you choose, the other answer becomes unavailable.

Dimensional modelling handles this explicitly with a **Type 2 slowly changing dimension**: the dimension keeps one row per version of the entity, with validity dates, and the fact points at the version that was true at the moment of the event.

■ SLOWLY CHANGING DIMENSION (TYPE 2) — DIM_CUSTOMER

CUST_KEY	CUST_ID	NAME	CITY	VALID_FROM	VALID_TO	CURRENT
4471	C001	Northline	Toronto	2024-01-01	2026-06-30	N
5820	C001	Northline	Vancouver	2026-07-01	9999-12-31	Y
4472	C002	Peel Supply	Mississauga	2023-05-14	9999-12-31	Y

Customer C001 moved to Vancouver on 1 July 2026. Orders before that date point at key 4471 (Toronto); orders after point at 5820 (Vancouver). Last year's regional numbers stay correct instead of being silently rewritten — something a flat table structurally cannot do.

Figure 9 — Type 2 SCD. Two rows, two surrogate keys, one real customer.

Why this matters commercially Orders placed before 1 July 2026 carry Customer_Key 4471 and roll up to Toronto. Orders after carry 5820 and roll up to Vancouver. Last year's regional revenue therefore stays exactly as it was reported, instead of being silently rewritten every time a customer moves. Sales territories, commission plans and year-on-year comparisons all depend on this behaviour.

Type 2 is the most common variant. Type 1 simply overwrites the attribute (use it when history genuinely does not matter, such as correcting a typo), and Type 3 keeps a single "previous value" column. The point is that dimensional modelling gives you an explicit choice; a flat table gives you none.

6.4 Conformed dimensions across multiple facts

No warehouse has only one fact table for long. After sales come returns, shipments, inventory, purchase orders, web sessions and support tickets.

If each is its own flat table, "customer segment" ends up defined six different ways, and the returns dashboard disagrees with the sales dashboard. Nobody can say which is right.

A **conformed dimension** is a single `Dim_Customer` shared by every fact table in the warehouse. Because all facts point at the same dimension rows, you can legitimately compare sales to returns by segment, or shipments to orders by vendor tier. This is the property that makes an enterprise warehouse an enterprise warehouse rather than a pile of unrelated extracts, and it is impossible with flat tables.

6.5 Maintainability and update anomalies

When a vendor rebrands in a flat table, you rewrite tens of millions of rows. If the job fails halfway, you now hold two different names for one vendor and every `GROUP BY` silently splits the totals. Nothing errors; the numbers are simply wrong.

In a star schema the same change is one row in `Dim_Vendor`. Adding a new attribute — vendor risk rating, customer loyalty tier — is one small column on a small table, immediately available to every historical fact row without reprocessing anything.

6.6 Compression and query performance

On a row-based engine, scanning a narrow fact table and joining a small dimension is far cheaper than scanning a table where every row carries hundreds of bytes of repeated text.

The effect is particularly pronounced in **Power BI**. The VertiPaq engine stores each column separately and dictionary-encodes its distinct values. A fact table of integer keys and measures compresses extremely well; a 100-million-row flat table with a dozen wide text columns does not compress nearly as effectively, inflates the .pbix file, slows refresh, and consumes far more memory in the service.

Note On modern columnar warehouses such as Snowflake, BigQuery and Databricks, columnar compression narrows this gap considerably, and joins are cheap. Performance alone is therefore a weaker argument than it was twenty years ago. The arguments in 6.2, 6.3 and 6.4 — consistency, history and conformity — are the ones that still hold unconditionally.

6.7 A clean semantic model for business users

Power BI, Tableau, Looker and every modern semantic layer are built on the assumption of facts and dimensions. Relationships flow one-way from dimension to fact; filters propagate down that path; measures aggregate the fact.

Given a proper star, a business user can drag "Vendor Country" against "Total Revenue" and get the right answer without writing a line of SQL or understanding the underlying tables. Given a 60-column flat table, they must know which columns are safe to sum, which are safe to group by, and which will double-count — knowledge that inevitably lives with one person who is on holiday.

6.8 Scalability

A flat table grows in two directions at once: more rows as business happens, and more columns as new attributes are requested. Both are expensive.

A star schema grows in one direction only. The fact table grows with events; dimensions grow only when a genuinely new entity appears. Adding the twentieth vendor attribute costs nothing at the fact level, and adding a fifth dimension is a single new key column.

7. Types of fact table

Not every fact table is a stream of transactions. Recognising which of the four types you are building prevents a class of aggregation bug.

■ TYPES OF FACT TABLE

TYPE	DESCRIPTION	EXAMPLE
Transaction	One row per event, as it happens. Fully additive.	Every sales line, every payment
Periodic snapshot	One row per entity per period. Semi-additive — do not sum across time.	Daily stock on hand, monthly balance
Accumulating snapshot	One row per process instance, updated as milestones complete.	Order → Picked → Shipped → Delivered
Factless fact	Records that an event occurred; no numeric measure.	Promotion coverage, student attendance

Figure 10 — The four fact table types.

Watch for semi-additive measures A periodic snapshot such as daily stock on hand can be summed across products and warehouses, but must never be summed across time — you would be adding Monday's stock to Tuesday's stock. Use an average or a closing balance instead. This class of error is invisible in a flat table because nothing tells you the measure is semi-additive.

8. The snowflake schema

A snowflake schema is a star schema in which the dimensions have themselves been normalised into sub-tables. Instead of `Dim_Product` carrying Brand and Manufacturer directly, it carries `Brand_Key` pointing at a separate `Dim_Brand`.

■ SNOWFLAKE SCHEMA — DIMENSIONS NORMALISED INTO SUB-TABLES

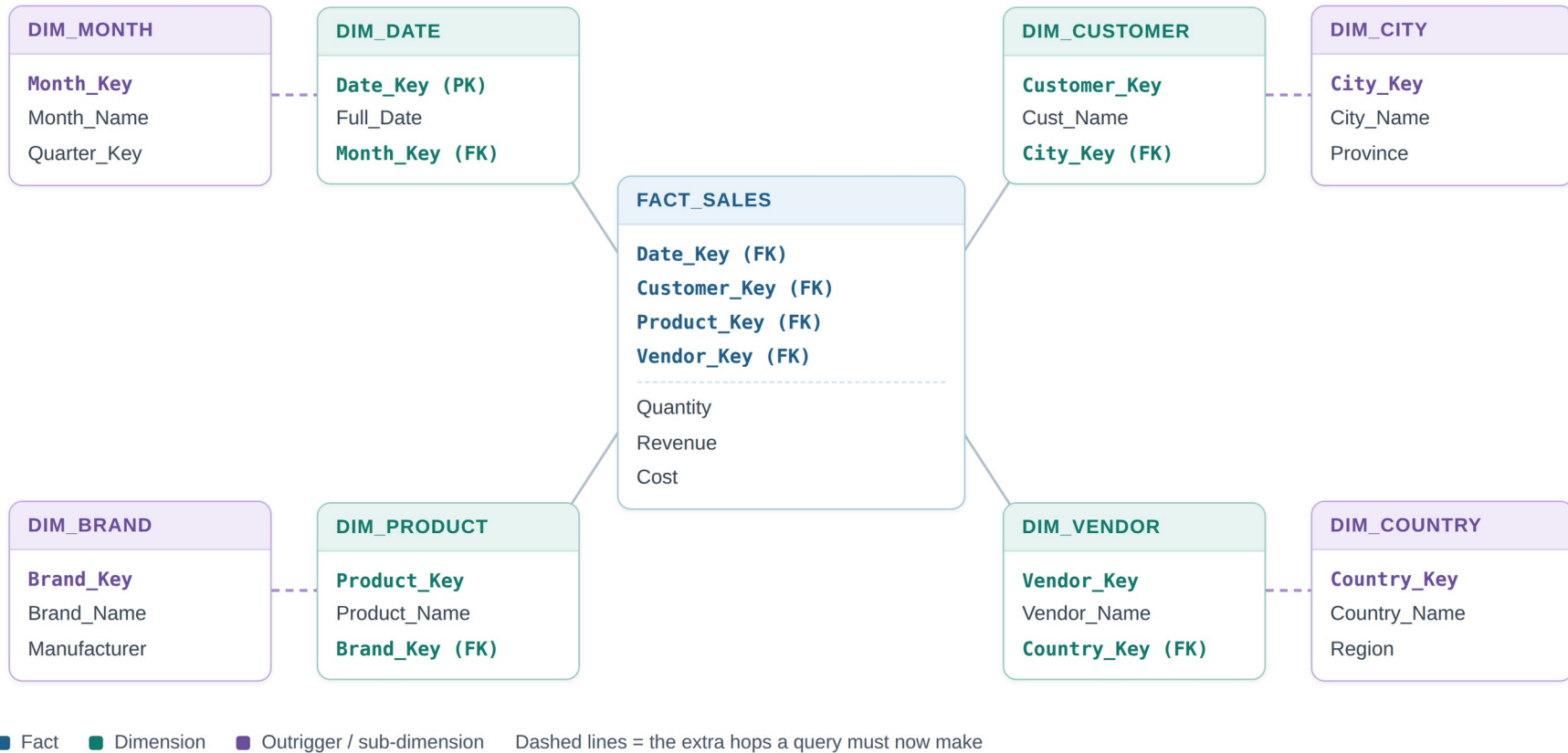


Figure 11 — A snowflake schema. Dimensions are normalised to third normal form, adding join hops.

8.1 When snowflaking is justified

- The dimension is genuinely enormous — tens of millions of rows — and the repeated attribute is large.

- A sub-entity is shared by several dimensions and must be maintained in one place (a geography hierarchy used by both customer and vendor).
- The sub-entity changes on a completely different cadence from its parent.
- Governance or regulation mandates third normal form in the presentation layer.

8.2 When it is not

- Purely to "save space" — modern columnar compression already handles repeated text well.
- Because it looks tidier on a diagram. Every extra hop is a join the query engine must resolve and a relationship a business user must understand.
- In a Power BI model, where a snowflaked dimension means more relationships, more ambiguity and slower filter propagation than a flattened one.

Default position Start with a star. Snowflake a specific dimension only when you can name the concrete problem it solves. A star schema with deliberately denormalised dimensions is the mainstream choice for analytics, and the one Power BI is optimised for.

9. Star vs. snowflake vs. one big table

■ STAR VS. SNOWFLAKE VS. ONE BIG TABLE

	ONE BIG TABLE (FLAT)	STAR SCHEMA	SNOWFLAKE SCHEMA
Structure	Everything in one table	Fact + denormalised dimensions	Fact + normalised dimensions
Normal form	ONF — fully denormalised	~2NF on dimensions	3NF on dimensions
Joins per query	None	1 hop per dimension	2–4 hops per dimension
Redundancy	Very high	Low	Lowest
Query performance	Fast on columnar; heavy scans	Fast & predictable	Slower — more joins
Ease for business users	Confusing when very wide	Highest	Lower — must know the chain
Supports history (SCD2)	No	Yes	Yes
Conformed across facts	No	Yes	Yes
Power BI / Tableau fit	Workable, poor compression	Native — the intended model	Works, but more relationships
Best used for	A single stable report or ML feature set	The enterprise analytics layer	Very large or volatile dimensions

Figure 12 — Side-by-side comparison of the three approaches.

10. The same question, both ways

It is worth being honest about the cost of the star schema: the SQL is longer.

■ THE SAME QUESTION, BOTH WAYS

FLAT TABLE	STAR SCHEMA
<pre>SELECT Vendor_Name, SUM(Revenue) AS Total FROM Orders WHERE Order_Date >= '2026-01-01' GROUP BY Vendor_Name;</pre>	<pre>SELECT v.Vendor_Name, SUM(f.Revenue) AS Total FROM Fact_Sales f JOIN Dim_Vendor v ON v.Vendor_Key = f.Vendor_Key JOIN Dim_Date d ON d.Date_Key = f.Date_Key WHERE d.Calendar_Year = 2026 GROUP BY v.Vendor_Name;</pre>

The star version is three lines longer — and it is the one that stays correct when a vendor is renamed, a customer relocates, or a second fact table is added. That trade is the entire argument.

Figure 13 — Revenue by vendor for 2026, expressed against each model.

Three extra lines buy the following: the vendor name cannot be split by a typo; the 2026 filter can use fiscal year, quarter, week or holiday flag without any of those columns existing on the fact; the same `Dim_Vendor` can be reused by the returns and shipments facts; and if the vendor is renamed next year, this query's historical output does not change. That is the trade, and in an analytics platform it is almost always worth making.

11. Choosing a model

■ CHOOSING A MODEL

IF THIS IS TRUE...	...USE THIS
Multiple fact processes share the same entities (sales, returns, shipments)	Star schema
Attributes change over time and history must stay correct	Star schema with SCD Type 2
Business users self-serve in Power BI or Tableau	Star schema
A dimension is enormous or highly volatile (e.g. 50M products)	Snowflake that dimension only
Strict 3NF is mandated by governance or regulation	Snowflake schema
One stable dashboard, one team, attributes never change	Flat table is acceptable
Feeding a machine-learning model that wants one wide row	Flat table built from the star

Figure 14 — Decision guide.

A flat table is a legitimate engineering choice when the data never changes, there is one fact process, and there is one consumer. The moment any of those three stops being true — a changing attribute, a second fact table, a second team — the flat table starts producing numbers that disagree with each other.

12. How the two approaches fit together

In current practice the choice is not star **or** flat. It is star **then** flat.

■ HOW THE TWO APPROACHES FIT TOGETHER IN PRACTICE



The star schema is the source of truth; the flat table is a materialised convenience built on top of it. Modern practice is not star or flat — it is star then flat.

Figure 15 — The star schema as governed core; wide tables as a disposable serving layer.

The star schema is where conformity, history and the single definition of each business entity are enforced. Wide denormalised tables are then materialised on top of it for a specific dashboard, extract or machine-learning feature set. Because those wide tables are generated from the star, they can be dropped and rebuilt at any time without losing anything — they are a cache, not a source of truth.

13. Common mistakes checklist

1. Building the fact table before declaring the grain in one sentence.
2. Splitting on data type — putting every number in the fact and every string in a dimension.
3. Using the source system's natural key in the fact instead of a surrogate key, which makes Type 2 history impossible.
4. Storing descriptive attributes on the fact table "for convenience". They will drift out of step with the dimension.
5. Creating a separate customer dimension per fact table instead of conforming one.
6. Summing a semi-additive snapshot measure across time.
7. Snowflaking every dimension by reflex, then wondering why the Power BI model is slow.
8. Forgetting an "unknown" member (key 0 or -1) in each dimension, so rows with missing keys are silently dropped by inner joins.
9. Building a date dimension with only a date column — the value is in the fiscal, holiday and period-offset attributes.

14. Glossary

Grain — The precise meaning of one row in a fact table. Declared before design begins.

Fact table — Table of business events at a defined grain, holding measures and foreign keys.

Dimension table — Table of descriptive attributes used to filter, group and label facts.

Measure — A numeric value in a fact table intended to be aggregated (revenue, quantity, cost).

Additive measure — A measure that can be summed across every dimension, including time.

Semi-additive measure — A measure summable across some dimensions but not time — typically balances and stock levels.

Surrogate key — A meaningless integer generated by the warehouse to identify a dimension row. Enables Type 2 history.

Natural / business key — The identifier used by the source system, e.g. C001. Retained in the dimension for traceability.

Degenerate dimension — An identifier held in the fact table with no dimension of its own, e.g. Order_ID.

Conformed dimension — A dimension shared identically by multiple fact tables, enabling cross-process comparison.

SCD Type 1 — Overwrite the attribute. No history retained.

SCD Type 2 — Add a new dimension row with validity dates. Full history retained.

SCD Type 3 — Keep a "previous value" column alongside the current one. Limited history.

Outrigger — A secondary dimension joined to another dimension rather than to the fact. The building block of a snowflake.

Star schema — One fact table joined directly to denormalised dimensions. One join hop per dimension.

Snowflake schema — A star whose dimensions are normalised into sub-tables. Multiple join hops per dimension.

One Big Table (OBT) — A fully denormalised wide table, typically materialised from a star for a single purpose.

Factless fact table — A fact table recording that an event occurred, with no numeric measure.

VertiPaq — The in-memory columnar engine behind Power BI and Analysis Services Tabular.

End of document.